Class 35 Everything About Malware Analysis For SOC

Malware Analysis

Definition: Malware analysis is the process of examining malicious software to understand its behavior, origin, structure, and potential threats.

Main goals of a Malware Analyst:

1. What happened? — Identify the malicious activity.
2. How did it get in? — Find the infection method.
3. What can it do? — Determine its capabilities and impact.
4. How do we detect/stop it? — Develop ways to detect, contain, and prevent it.

Fundamentals: Malware & Analysis

- Malware (Malicious Software): Software or code designed to harm systems, steal data, disrupt services, demand ransom, or conduct espionage.
- Malware Analysis: Examining and reverse engineering malware to understand how it works and extract useful security information.

Key Terms

- IOC (Indicator of Compromise): Evidence that a system may have been compromised. Examples: malicious IP addresses, domains, file hashes, and suspicious registry keys.
- TTPs (Tactics, Techniques, and Procedures): Describes how an attacker operates.
- Tactics: The attacker's goal.
- Techniques: The method used to achieve the goal.
- Procedures: The specific steps used to carry out the method.

Purpose: Malware analysis helps security teams improve detection rules, strengthen defenses, and respond effectively to cyberattacks.

Taxonomy Of Malware Types

Type	Core Mechanism	Objective
Ransomware	Encrypts local/network files using asymmetric key cryptography.	Extorts victims for decryption keys.
RAT (Remote Access Trojan)	Establishes an interactive, encrypted Command & Control (C2) channel.	Allows full remote control, keylogging, and shell access.
InfoStealer	Scrapes browser SQLite databases, crypto wallets, and system credentials.	Siphons sensitive credentials for sale or initial access.
Worm	Exploits network vulnerabilities to self-propagate automatically.	Spreads rapidly without human interaction across flat networks.
Rootkit	Operates at the kernel level (Ring 0) or hypervisor level.	Hooks OS API calls to hide processes, files, and network sockets.
Botnet Client	Joins an infected host to a centralized network overlay.	Executes coordinated tasks like DDoS or mass spam distribution.

Types of Malware Executables

Malware executables are malicious programs that run on a computer. They can be classified into two main types: Binary Malware and Script Malware.

1. Binary Malware

- Definition: Malware compiled into native machine code that the processor can execute.
- Architecture-bound: Often designed for a particular CPU architecture and operating system.
- Analysis: Usually requires assembly-language knowledge and reverse-engineering tools.
- Examples: `.EXE`, `.DLL`, `.SYS` files on Windows.

2. Script Malware

- Definition: Malicious code written in a scripting language and executed by an interpreter.
- Interpreter-dependent: Requires a suitable runtime or interpreter, such as PowerShell or JavaScript.
- Analysis: Analysts examine the script's code, behavior, and any obfuscation used to hide its purpose.
- Examples: `.PS1` (PowerShell), `.JS` (JavaScript), `.VBS` (VBScript).

Key Differences

Feature	Binary Malware	Script Malware
Form	Compiled machine code	Script source code or encoded text
Execution	Executed by the operating system/processor	Executed through an interpreter or runtime
Analysis	Often requires disassembly and reverse engineering	Often involves reading and deobfuscating code
Examples	EXE, DLL, SYS	PS1, JS, VBS

Remember: Binary malware is generally compiled into machine code, whereas script malware relies on an interpreter or runtime. Both types can use obfuscation to make analysis harder.

1. Binary Malware

Definition: Malware distributed as compiled executable files containing machine code that can run on a compatible system.

- File types: `.exe`, `.dll`, `.sys`, ELF.
- Execution: Runs as native machine instructions through the operating system.
- Delivery methods: Email attachments, malicious downloads, USB drives, or other malware.
- Identification: File hashes (e.g., MD5) and unique byte sequences or strings can help identify malware, even if its filename changes.
- Analysis tools: Ghidra, IDA Pro (disassembly/reverse engineering), and x64dbg (debugging).

Example: An attacker disguises an executable as `invoice.pdf.exe` to trick a user into running it.

2. Script-Based Malware

Definition: Malware written in a scripting language that relies on an interpreter or runtime to execute its instructions.

- File types: `.ps1`, `.vbs`, `.js`, `.bat`, `.py`.
- Execution: Uses interpreters such as `powershell.exe` or `wscript.exe`.
- Delivery methods: Email attachments, Office documents with malicious macros, and compromised websites.

- Modification: Scripts are often easier to modify and adapt than compiled binaries.
- Analysis: Requires deobfuscation, such as decoding Base64, resolving string concatenation, reversing XOR operations, and understanding renamed variables.

Example: A malicious Word document (`report.docm`) runs a macro that launches a hidden PowerShell script.

3. Key Differences

Feature	Binary Malware	Script Malware
Format	Compiled machine code	Script instructions
Execution	Native executable via OS	Interpreter or runtime
Analysis	Disassembly and debugging	Code inspection and deobfuscation
Common tools	Ghidra, IDA Pro, x64dbg	Text editor, script analysis tools
Modification	Usually requires recompilation to change program logic	Often easier to edit
Detection	Hashes, strings, byte patterns, behavior	Script content, decoded commands, behavior

Important Terms

- Checksum / Hash: A value calculated from file contents. MD5 can help identify a file, but it is cryptographically weak; SHA-256 is generally preferred for security investigations.
- Disassembly: Converting machine code into readable assembly instructions.
- Deobfuscation: Making intentionally hidden or confusing code understandable.
- Living off the land: Using legitimate system tools, such as PowerShell, to perform malicious actions.
- In-memory execution: Running code in memory without necessarily saving the payload as a file on disk.

Remember: Binary malware analysis focuses on machine code and program behavior, while script malware analysis focuses on script logic, interpreters, and deobfuscation. Both may require behavioral analysis to determine the actual threat.

High Profiles Real World Cases

Stuxnet (2010): The first known cyber weapon targeting physical infrastructure. It utilized four zero-day vulnerabilities and hijacked Siemens PLCs controlling Iranian nuclear centrifuges while faking normal telemetry data.

WannaCry (2017): A global ransomware outbreak that used the NSA's leaked **EternalBlue** exploit (SMBv1) to spread autonomously as a worm, causing widespread disruption to critical infrastructure, including the UK's NHS.

SolarWinds / SUNBURST (2020): A sophisticated supply chain attack where state-sponsored threat actors injected a backdoor directly into Orion software updates, compromising thousands of government and corporate networks globally.

Malware Analysis Pipeline

The malware analysis pipeline consists of three main approaches: Static Analysis, Dynamic Analysis, and Code Reversing. These methods help analysts understand a malware sample, identify its capabilities, and develop detection methods.

1. Static Analysis

Definition: Examining a malware file without executing it.

What analysts check:

- File hashes (SHA-256): Identify and compare files using a unique fingerprint.
- PE headers: Examine Windows executable metadata and file sections.
- Embedded strings: Find readable text, URLs, IP addresses, and suspicious commands.
- Import Address Table (IAT): Identify Windows API functions the program may use.
- YARA signatures: Search for patterns associated with known malware.

Tools: PEStudio, FLOSS, `pestr`, YARA.

Example: Finding a suspicious URL inside an executable without running the file.

2. Dynamic Analysis

Definition: Running malware in a controlled, isolated environment to observe its actual behavior.

What analysts monitor:

- Process tree: Which processes the malware creates.
- File system and registry: Files created or modified and registry changes.
- Network traffic (PCAP): Connections to suspicious IP addresses and domains.
- Memory dumps: Data and code present in memory during execution.
- API calls: Operating system functions used by the malware.

Tools: ProcMon, Wireshark, x64dbg, CAPE Sandbox, Volatility.

Example: Observing malware create a new process, modify a registry key, and connect to a suspicious domain.

Important: Dynamic analysis should be performed in a properly isolated malware-analysis lab, not on your normal computer.

3. Code Reversing (Reverse Engineering)

Definition: Examining compiled machine code to understand the program's internal logic and functionality.

What analysts do:

- Assembly analysis (x86/x64): Examine low-level CPU instructions.
- Disassembly: Convert machine code into readable assembly instructions.
- Debugging: Execute instructions step by step to understand program behavior.
- API analysis: Understand how the program interacts with the operating system.
- Breakpoint analysis: Pause execution at selected instructions or functions to inspect program state.

Tools: IDA Pro, Ghidra, x64dbg.

Example: Discovering that a malware function decrypts a configuration containing command-and-control server details.

Static vs. Dynamic Analysis

Feature	Static Analysis	Dynamic Analysis
Execution	Does not execute the sample	Executes the sample
Main focus	File structure and code	Actual runtime behavior
Detects	Strings, headers, imports, signatures	Network activity, file changes, processes
Main advantage	Can be performed without running malware	Reveals behavior that may be hidden statically
Limitation	Obfuscation can hide important details	Malware may detect the sandbox or behave differently

Important Terms

- PE (Portable Executable): The file format used by Windows executables and DLLs.
- IAT (Import Address Table): A structure that helps identify external functions imported by a PE file.
- PCAP: A file format used to store captured network traffic.
- YARA: A tool for identifying files based on specified patterns.
- Sandbox: An isolated environment for safely observing suspicious programs.
- Volatility: A framework for analyzing memory captures.
- API Hooking: Intercepting or monitoring function calls to observe program behavior.

Quick Revision

Remember the pipeline as:

Static Analysis → Dynamic Analysis → Code Reversing

- Static: What can the file tell us without running it?
- Dynamic: What does the malware actually do when it runs?

- Code reversing: How does the program's internal logic work?

These approaches complement one another. Analysts combine their findings to identify indicators of compromise (IOCs), understand attacker techniques, and improve security detections.

Anti-Analysis Techniques & Evasion

Definition: Anti-analysis techniques are methods used by malware authors to make malware harder to detect, debug, analyze, and reverse engineer.

1. Anti-Debugging

Meaning: Malware checks whether it is running inside a debugger and may change its behavior if one is detected.

- IsDebuggerPresent(): A Windows API that checks whether the calling process is being debugged.
- PEB (Process Environment Block): A Windows data structure that contains process information, including a `BeingDebugged` flag.
- JZ/JNZ: Conditional jump instructions in assembly that determine program flow based on CPU flags.

Example: Malware detects a debugger and exits instead of revealing its behavior.

Analysis concept: Analysts may inspect the relevant conditional branch or use specialized debugging tools, such as ScyllaHide, to investigate or bypass certain checks.

2. Anti-VM / Sandbox Evasion

Meaning: Malware attempts to detect whether it is running in a virtual machine (VM) or automated sandbox.

Common detection methods:

- VM artifacts: Checking for VMware or VirtualBox drivers, files, and registry keys.
- MAC address checks: Looking for network adapter address prefixes associated with virtual environments.
- Hardware checks: Detecting unusually low CPU core counts or limited system resources.

- Timing checks: Using `RDTSC` (Read Time-Stamp Counter) to measure execution time and identify delays that may indicate debugging or analysis.

Example: Malware detects a virtualized environment and remains inactive to avoid revealing its behavior.

Key point: These checks can produce false positives because legitimate computers may have similar characteristics.

3. Packing & Obfuscation

Meaning: Techniques used to conceal malware code and make analysis more difficult.

- Packing: Compresses or transforms an executable so its original code is not immediately visible.
- Encryption: Protects the payload using an encryption mechanism that must be reversed or decrypted.
- Obfuscation: Makes code difficult to understand without necessarily encrypting it.
- UPX: A commonly used executable packer that can also be used to package some malware.

How runtime unpacking works:

1. The packed executable starts running.
2. Its unpacking routine reconstructs the original code in memory.
3. Execution transfers to the OEP (Original Entry Point), where the original program begins executing.

Example: Static analysis reveals only an unpacking routine, while the actual malicious code becomes visible in memory after unpacking.

Quick Revision Table

Technique	Main purpose	Key indicators
Anti-debugging	Detect debuggers	`IsDebuggerPresent()`, PEB flag
Anti-VM / sandbox	Detect analysis environments	VM artifacts, hardware and timing checks

Technique	Main purpose	Key indicators
Packing	Hide the original executable code	Unpacking routine, modified file structure
Obfuscation	Make code difficult to understand	Encoded strings, confusing control flow

Remember: Anti-analysis techniques do not necessarily make malware undetectable. They make investigation more difficult or cause the malware to behave differently in an analysis environment. Malware analysts combine static analysis, dynamic analysis, and reverse engineering to recognize these techniques and understand the hidden behavior.

Essential Tools For Malware Analysis

Category	Tools	Primary Purpose
Static Inspection	CFF Explorer, PEStudio, Detect It Easy (DiE)	Inspecting PE structures, dependencies, sections, and signatures.
Disassemblers	IDA Free, Ghidra, Cutter	Transforming machine code into readable assembly or decompiled C-like code.
Dynamic Debuggers	x64dbg, x32dbg, WinDbg	Controlling execution step-by-step, setting breakpoints, and dumping memory.
System Monitoring	Process Monitor (ProcMon), Process Hacker	Tracking real-time file system, registry, process, and thread activities.
Network Interception	Wireshark, Fiddler, PolarProxy	Capturing, inspecting, and decrypting C2 network communication packets.